

UNIVERSIDAD DE LA SABANA

Departamento de Ingeniería

Maestría en Arquitectura de Software

Observabilidad en Ambientes Productivos

SEGUIR UN PEDIDO DE PUNTA A PUNTA

Laboratorio 3.3 · Informe ejecutivo del sistema integral de observabilidad, AIOps, seguridad de red, SRE y chaos engineering

Caso de laboratorio: flujo de pedidos con tres microservicios Spring Boot, pipeline OpenTelemetry local e infraestructura preparada para GKE y ECS

Fabián Andrés Rojas García

Jorge Eliécer Rojas Quiñones

Juan Esteban Gómez Roa

Bogotá, Colombia · 2026

1. Resumen ejecutivo

Cuando un flujo de pedidos distribuido se degrada, la persona de turno necesita saber tres cosas: qué servicio falló, si el problema es de latencia o de errores y cuánto presupuesto de error se está gastando. Una métrica suelta no responde ninguna de las tres. Para eso el mismo identificador de traza tiene que aparecer en el log y en el backend de trazas, y la alerta no puede saltar con el primer punto de una serie que apenas empieza.

Para el laboratorio montamos una cadena de tres procesos Spring Boot. `service-a` recibe `GET /api/orders/{orderId}`, `service-b` prepara el cumplimiento y puede inyectar una espera de 200 ms, y `data-service` es el único que abre JDBC contra MySQL. El agente Java 2.31.1 instrumenta HTTP y el driver; en el código agregamos tres spans de negocio. Un Collector 0.162.0 manda las trazas a Tempo, las métricas a Prometheus y los logs a Loki, y Grafana arranca con dos tableros aprovisionados. En Prometheus quedaron umbrales fijos y una regla correlacionada que solo dispara cuando el error rate supera su baseline más dos desviaciones estándar y, a la vez, el p99 de `service-a` pasa de 150 ms.

El estado al 9 de octubre de 2026 es este. `mvn test` terminó con código 0, y las pruebas cubren las compuertas del caos, la espera de 200 ms, el error inyectado antes de tocar la base y la lectura del pedido ord-001 sobre H2. Lo que no logramos fue levantar el stack con Docker: el registro de imágenes respondió 504 al autorizar la descarga de `maven:3.9.9-eclipse-temurin-21` y de `eclipse-temurin:21-jre-jammy`. Por eso aquí no hay traza de Tempo, captura de Grafana, MTTD medido, conteo de alertas en firing ni porcentaje de reducción de ruido. Tampoco se hizo apply de Terraform.

Para que ningún archivo YAML se lea como si fuera una ejecución, cada afirmación del informe cae en una de tres clases:

Clase	Ejemplos en este informe	Cómo se comprueba
Verificado	Compilación y pruebas Maven, fórmula del error budget, compuertas del caos	`mvn test`, ejecutado el 9 de octubre de 2026
Implementado, sin ejecución de punta a punta	Traza en Tempo, alerta en firing, MTTD, tableros con series	PASOS.md, secciones B y C, cuando el registro de imágenes responda
Preparado para cloud, sin desplegar	GKE, Cloud SQL, ECS, RDS, flow logs, Service Connect, Cloud Service Mesh	`terraform plan`; no hubo apply

Tabla 1. Clases de evidencia usadas en todo el documento (elaboración propia).

Capacidad	Estado
Tres servicios y regla de stock	Implementada. Probada en unidad con H2, no contra MySQL en contenedor.
Agente, spans de negocio y logs JSON	Implementados en imagen y código. El pipeline completo no se vio corriendo.
Regla baseline + 2σ y alertas estáticas	Escritas en Prometheus. Ninguna llegó a firing.
Enriquecimiento con trace_id	Script listo. No se probó contra una alerta real.
Caos de 200 ms y de 10 %	Implementado y apagado por defecto. Las compuertas pasaron en pruebas unitarias; no hubo corrida contra Prometheus.
SLO de 99 % y fórmula del presupuesto	Fórmula probada con 1000 peticiones y 10 fallos. Falta una ventana experimental medida.
GKE, Cloud SQL, ECS, RDS, flow logs	Versionados, no creados.
Service mesh L7	App Mesh descartado por fin de soporte. Cloud Service Mesh y Service Connect documentados, sin aplicar.

Tabla 2. Capacidades y estado de verificación al cierre del laboratorio (elaboración propia).

2. Objetivos y alcance

2.1 Objetivo general

Construir un laboratorio en el que una solicitud de pedido se pueda seguir desde la entrada HTTP hasta la sentencia SQL, y en el que una degradación controlada se detecte, se explique y se revierta sin quedar encendida por descuido.

2.2 Objetivos específicos

- Instrumentar los tres servicios con el agente Java y con spans de negocio que no dupliquen el span JDBC.
- Propagar el contexto W3C entre `service-a`, `service-b` y `data-service`.
- Exportar métricas, logs y trazas por OTLP hacia un Collector que los reparta a backends distintos.
- Definir una anomalía que exija dos condiciones a la vez: error rate estadístico y latencia p99 por encima del SLO.
- Separar la telemetría de aplicación, la de proxies y la de flujo de red, sin presentar como desplegado un mesh que no se aplicó.
- Dejar dos experimentos reversibles, apagados por defecto, y una fórmula explícita de error budget.
- Calificar ocho dominios de madurez solo con evidencia verificada.

2.3 Alcance funcional

El caso de negocio es consultar un pedido y decidir si el inventario alcanza. La semilla local carga los pedidos ord-001 a ord-005; ord-001 corresponde al SKU LAPTOP-X1. La respuesta dice si el stock es suficiente y, cuando hay un span válido, trae el `traceId`. La autenticación del laboratorio es un único endpoint `POST /api/session`: un token distinto del configurado recibe 401 y suma al contador de fallos. No pretende ser un proveedor de identidad.

2.4 Limitaciones

- El entorno local usa MySQL 8.4 en Docker, que no reemplaza a Cloud SQL ni a Amazon RDS.
- Cada proceso de `data-service` activa un solo perfil Spring (local, gcp o aws); nunca abre las dos bases cloud a la vez.
- No hay escáner de CVE conectado. El tablero de seguridad lo indica en un panel de texto y no muestra ningún número.
- Security Command Center y Security Hub no están activados, y sus hallazgos no se simulan.
- H2 en las pruebas de Maven no sustituye al MySQL del compose.
- Sin credenciales cloud y con el registro de imágenes caído, no podemos afirmar un despliegue ni un MTTD.

3. Arquitectura observable

`service-a` es la única puerta de entrada del flujo y no tiene URL de base de datos. Llama a `service-b` en `GET /api/fulfillment/{orderId}`. Si el experimento de latencia está armado y encendido, `service-b` duerme 200 ms dentro del span `fulfillment.prepare` y luego llama a `data-service` en `GET /api/records/orders/{orderId}`. `data-service` ejecuta el JOIN entre orders e inventory y compara la cantidad disponible con la pedida. Cuando el experimento de error está activo y la petición trae la cabecera de laboratorio, el fallo ocurre antes de la consulta.



Figura 1. Recorrido de una consulta de pedido. Los puertos son los publicados en el host (elaboración propia).

Servicio	Responsabilidad	Persistencia
service-a	Coordina el pedido, expone la sesión y reenvía la cabecera de experimento.	Ninguna
service-b	Prepara el cumplimiento. Es el único lugar donde vive la espera de 200 ms.	Ninguna
data-service	Lee pedido e inventario. Es el único lugar del error inyectado y del JDBC.	MySQL, una URL por perfil

Tabla 3. Responsabilidad de cada proceso (elaboración propia).

3.1 Perfiles de base de datos

El perfil local es el del compose y también el del MySQL que el manifiesto de GKE despliega dentro del clúster; su URL sale de `SPRING_DATASOURCE_URL`. El perfil gcp no tiene valor por defecto: exige `CLOUD_SQL_JDBC_URL` y solo tiene sentido con `create_cloud_sql` aplicado. El perfil aws exige `RDS_JDBC_URL`. Si `create_rds` sigue en false, esa URL apunta al MySQL que viaja en la misma tarea de ECS. Ese sidecar no es un RDS simulado desde el portátil, y el MySQL del compose tampoco es Cloud SQL.

Perfil	Variable de URL	Cuándo se usa
local	`SPRING_DATASOURCE_URL`	Compose y MySQL del manifiesto de GKE
gcp	`CLOUD_SQL_JDBC_URL`	Solo con Cloud SQL creado; sin fallback local
aws	`RDS_JDBC_URL`	RDS si `create_rds` es true; si no, el sidecar de la tarea

Tabla 4. Perfiles de base de datos. Un proceso nunca combina gcp y aws (elaboración propia).

3.2 Pipeline de señales

Cada proceso exporta OTLP gRPC a `otel-collector:4317`. El muestreo de cabeza es `parentbased_traceidratio` con argumento 1.0, porque en un laboratorio queremos todas las trazas. El Collector limita la memoria a 400 MiB, agrega el entorno local y el namespace lab33, descarta los spans de salud y los de `/api/chaos`, agrupa en lotes y exporta (OpenTelemetry, s.f.-b). Las trazas salen por `otlp_grpc/tempo`. Los logs pasan por un transform que escribe `trace_id=` en el cuerpo y salen por `otlp_http/loki`. Las métricas se exponen en el puerto 8889 con etiquetas constantes de service.name, versión, entorno, namespace y cloud.provider, y Prometheus las scrapea. Grafana lee de Prometheus, Tempo y Loki.



Figura 2. Camino de las tres señales. El Collector no instrumenta la aplicación: recibe y reparte (elaboración propia).

Componente	Versión fijada
Java y Spring Boot	21 y 3.5.16
API de OpenTelemetry en el jar	1.65.0 (el SDK de runtime lo pone el agente, no una segunda dependencia)
Agente Java	2.31.1
Collector contrib	0.162.0
MySQL, Tempo y Loki	8.4, 2.8.1 y 3.4.2
Prometheus y Grafana	v3.4.1 y 11.6.0

Tabla 5. Versiones fijadas en el pom, el Dockerfile y el compose local.

4. Instrumentación OpenTelemetry

El jar depende solo de la API. Si además metíamos el SDK junto al agente, habría providers duplicados. `entrypoint.sh` agrega `-javaagent` únicamente cuando `OTEL_ENABLED` no es false, de modo que el mismo artefacto puede arrancar sin telemetría. No usamos esa vía para medir overhead porque no corrimos ningún benchmark.

4.1 Qué abre el agente y qué abre el código

El agente abre el span servidor de Tomcat y Spring MVC, el span cliente de la llamada saliente y el span de base de datos sobre el driver JDBC. Estos spans siguen las convenciones semánticas que trae la versión 2.31.1 (sistema, nombre de operación y sentencia), así que el código no tiene por qué copiar el SQL en un span manual (OpenTelemetry, s.f.-a, s.f.-c). Escribimos a mano tres spans: `order.coordinate` en `service-a`, `fulfillment.prepare` en `service-b` y `record.load` en `data-service`. Los tres nombran una decisión de negocio, y ninguno lleva `@WithSpan` encima de esos mismos bloques.

Las rutas de actuator y de `/api/chaos` están en `OTEL_JAVAAGENT_EXCLUDE_URLS`, y el filtro del Collector repite el descarte. La idea es que el control del experimento no contamine el SLI de `/api/orders/` ni el error rate de `/api/records/`.

4.2 Métricas, logs y recursos

El agente exporta el histograma `http.server.request.duration`, las métricas de JVM y los contadores que registra el código: fallos de autenticación, inyecciones de latencia e inyecciones de error. El intervalo de exportación es de 15 segundos. No hay un segundo scrape de Micrometer en `/actuator/prometheus`; el actuator solo expone salud y Prometheus lee del Collector.

Logback escribe JSON con nivel, servicio, entorno y, cuando hay un span válido en el contexto, `trace_id` y `span_id`. El recurso del proceso fija `service.namespace=lab33`, `service.version=1.0.0`, `deployment.environment=local` y `cloud.provider=local`. En los manifiestos cloud esas etiquetas cambian según el proveedor, pero no las hemos visto en un clúster real.

4.3 Propagación

`OTEL_PROPAGATORS` vale `tracecontext,baggage`. El agente escribe la cabecera `traceparent` en el cliente HTTP y el servicio siguiente extrae el mismo `trace_id` (W3C, 2021). Además, `service-a` reenvía la cabecera `X-Lab-Experiment` cuando la petición de entrada la trae. Esa cabecera no es contexto de traza; es la marca que limita el error del 10 % a solicitudes identificables.

5. Correlación entre métricas, logs y trazas

Correlacionar exige que el mismo identificador de 32 caracteres hexadecimales aparezca en la respuesta, en el log y en Tempo, y que Grafana tenga un enlace capaz de leerlo. Imprimir un UUID que no viaja en la cabecera W3C no sirve para nada.

5.1 Cómo nace el identificador

El agente crea el `trace_id` cuando `service-a` recibe la petición. `order.coordinate` hereda ese contexto porque se abre en el hilo activo, y `RestClient` no arma la cabecera a mano. `service-b` y `data-service` continúan la traza. La respuesta JSON incluye `traceId` para poder copiarlo sin abrir los logs. Si aparecen dos `span_id` distintos con el mismo `trace_id`, la cadena avanzó.

5.2 Del log a la traza y de vuelta

El processor `transform/logs` copia el identificador al atributo y al cuerpo con el prefijo `trace_id=`. El datasource de Loki tiene un derived field con la expresión `trace_id=([0-9a-f]{32})` que apunta a Tempo. Un detalle que nos costó: en el YAML de aprovisionamiento el dólar se escribe `$$`, porque Grafana sustituye `${...}` por variables de entorno al cargar el archivo. Con un solo dólar la URL queda vacía y el salto desaparece sin dar error (Grafana Labs, s.f.-a). Tempo

declara `tracesToLogsV2` hacia Loki, filtrando por el trace id de la traza abierta con una ventana de dos minutos. Jaeger no está en el compose.

5.3 De la métrica a la traza

El exporter Prometheus del Collector tiene OpenMetrics activado y Prometheus arranca con `exemplar-storage`. El datasource declara el destino del exemplar hacia Tempo, lo que deja preparado el salto desde un punto del gráfico. Pero mientras no veamos el exemplar en Explore no podemos decir que el agente lo esté adjuntando. Lo que el código sí garantiza, con el stack arriba, es copiar el `traceId` de la respuesta y buscarlo en Loki y en Tempo.

Con este diseño, una investigación arranca así:

1. Leer la alerta y su etiqueta `anomaly_type`.
2. Si es de latencia, revisar el p99 de `/api/orders/` y el span `fulfillment.prepare`, que lleva `chaos.delay_ms` cuando la espera fue inyectada.
3. Si es de error, buscar en Loki la línea de error inyectado de `data-service`.
4. Correr `observability/aiops/enrich_incident.py`, que lee las alertas de Prometheus y adjunta el `trace_id` si Loki tiene una línea reciente. Si no la encuentra, el incidente sigue siendo válido y el script lo dice.

Prometheus no guarda el identificador en la serie, así que meterlo en la regla PromQL no es posible con estas métricas. La secuencia está implementada pero no la demostramos con una petición real. La prueba pendiente es un GET de ord-001 con el mismo `traceId` en los logs de los tres servicios, en Loki y en Tempo, y con el span JDBC debajo de `record.load`.

6. AIOps y detección de anomalías

Cloud Monitoring puede detectar anomalías sobre series que ya estén en ese servicio, y Amazon DevOps Guru analiza los recursos de AWS que se le habiliten. Ninguno de los dos estuvo disponible: no tenemos proyecto con facturación ni hicimos apply. La detección que sí se puede reproducir es la de Prometheus, y sus reglas quedan como el contrato que un adaptador cloud tendría que respetar.

6.1 Definiciones de la regla

Término	Definición en el laboratorio
`error_rate`	Cociente entre respuestas 5xx y peticiones totales de `data-service` en `/api/records/`, con rate de 2 minutos. Sin 5xx, el numerador es 0.
baseline	`avg_over_time` de ese cociente durante 10 minutos.
σ	`stddev_over_time` del mismo cociente durante 10 minutos.
p99	`histogram_quantile(0.99)` del histograma de `service-a` en `/api/orders/`, con rate de 2 minutos.
Umbral de latencia	150 ms. Es un umbral de diseño que la inyección de 200 ms debe cruzar, no un p99 medido.
SLO de éxito	99 % de respuestas 2xx en `/api/orders/`.
Historia mínima	`count_over_time` de la serie de error en 10 minutos ≥ 8, y desviación igual a sí misma para descartar NaN.

Tabla 6. Términos de la regla correlacionada. Diez minutos de baseline evitan una desviación inestable (elaboración propia, con funciones de Prometheus, s.f.).

La alerta `DataServiceCorrelatedAnomaly` dispara solo si se cumplen las cuatro condiciones a la vez: exceso sobre baseline + 2σ, p99 de `service-a` por encima de 150 ms, al menos ocho muestras y desviación definida. Ojo con un detalle de PromQL: en un `and` se conserva el valor del lado izquierdo, así que la anotación `error_rate` muestra el cociente de error y no la latencia. La latencia va en la anotación `slo_seconds`, fija en 0,150.

La alerta lleva servicio, entorno, plataforma, tipo de anomalía, severidad `critical` y una recomendación: revisar si el caos sigue activo y después la traza del error inyectado y el SELECT. El identificador de incidente lo arma el script de

enriquecimiento al leer la alerta; la regla de Prometheus no genera ningún UUID. El enlace operativo son las consultas de Loki y Tempo anotadas en la propia alerta.

6.2 Umbral fijo frente a correlación

Conviven tres alertas de confiabilidad:

- `ServiceALatencySlo`: p99 mayor que 150 ms durante 30 segundos.
- `DataServiceErrorStatic`: error rate mayor que 5 % durante 30 segundos.
- `DataServiceCorrelatedAnomaly`: las dos familias de síntoma más la historia mínima.

El diseño permite tres experimentos sobre el mismo tráfico y las mismas ventanas: solo latencia, solo error y combinado. Nuestra hipótesis es que la latencia sola enciende la alerta estática de SLO y no la correlacionada, porque el error rate sigue cerca de cero; que los errores solos encienden el umbral del 5 % sin cruzar 150 ms si `data-service` responde rápido; y que solo la combinación cumple la regla estadística.

Esa comparación no se ejecutó y no tenemos un archivo de resultados con conteos, así que no reportamos ninguna reducción porcentual de ruido. El procedimiento para obtenerla está en `chaos/run_scenarios.py`: tres pasadas con un tope de 150 segundos cada una y un JSON que solo se escribe si el stack responde. Hasta que ese JSON exista, la comparación es un diseño y no un resultado.

La alerta de autenticación es aparte: `increase(auth_failures_total[5m]) > 3`. El contador sube cuando se rechaza el token, no por un log fabricado.

7. Observabilidad de red y de seguridad

Distinguimos cuatro capas que no se deben mezclar:

- Telemetría de aplicación: la del agente. Es la única que existe hoy.
- Telemetría L7 de proxy: saldría de Cloud Service Mesh o de ECS Service Connect. Ninguno está aplicado.
- Telemetría de red: en cloud serían los VPC Flow Logs, escritos en Terraform y no creados.
- Eventos de seguridad de plataforma: Security Command Center y Security Hub. Requieren activación, permisos y, en el caso de Hub, servicios que generen hallazgos. No están activos y no les atribuimos ningún hallazgo.

7.1 Por qué no hay mesh desplegado

AWS App Mesh dejó de aceptar clientes nuevos el 24 de septiembre de 2024 y su soporte terminó el 30 de septiembre de 2026 (Amazon Web Services, s.f.-a), así que lo dejamos fuera. La alternativa vigente en ECS es Service Connect (Amazon Web Services, s.f.-b). No la incluimos en la definición del servicio porque la tarea del laboratorio pone los tres procesos en la misma red awsvpc y se hablan por 127.0.0.1, y ese tráfico no pasa por un proxy. Separar cada proceso en su propio servicio ECS cuesta tareas adicionales y queda como paso posterior.

En GKE, Cloud Service Mesh es el mesh gestionado basado en Istio (Google Cloud, s.f.-a). No está en el kustomization porque inyectar sidecars sin el fleet configurado deja los pods sin arrancar. Mientras tanto, el tráfico este-oeste local se observa con `http_client_request_duration` del agente.

7.2 Qué muestra el tablero y de dónde sale

Panel	Fuente real	Qué no representa
Autenticaciones rechazadas	Contador `auth.failures` y respuestas 401 de `POST /api/session`	No es Cloud Audit ni CloudTrail
Tráfico norte-sur	Histograma HTTP servidor de `service-a`	No son flow logs de la VPC
Tráfico este-oeste	Histograma HTTP cliente de `service-a` y `service-b`	No son métricas de Envoy
Errores 401, 403 y 5xx de cliente	Códigos del histograma del agente	No son hallazgos de Security Hub

Panel	Fuente real	Qué no representa
CVE	Panel de texto: no hay fuente	No muestra ningún número

Tabla 7. Cada panel del tablero de seguridad declara su fuente. Sin el stack corriendo, los paneles no tienen series observadas.

El rechazo de sesión no escribe el token. La comparación se hace en tiempo constante y el log solo dice "autenticacion rechazada". Las contraseñas de MySQL viven en variables de entorno del compose y en un ejemplo de secreto de Kubernetes que no debe aplicarse tal cual; ninguna aparece en los logs de aplicación.

En GCP, el Terraform de la subred incluye flow logs con muestreo de 0,5 (Google Cloud, s.f.-b). Cloud SQL está detrás de `create_cloud_sql`, en false, con IP privada y protección contra borrado. Activarlo sin el emparejamiento de Service Networking haría fallar el apply, por eso no lo damos por utilizable. En AWS, `flow_logs.tf` envía los flujos de la VPC a un log group propio, con un rol que solo puede asumir el servicio de flow logs (Amazon Web Services, s.f.-c). CloudTrail y Security Hub quedan como requisitos de una cuenta de laboratorio; crearlos tiene costo y no lo hicimos.

8. Chaos engineering, MTTD, SLO y error budget

Los dos experimentos arrancan apagados. `CHAOS_ARMED` es false en el compose base, así que aunque alguien llame al `POST` de latencia o de errores, la respuesta es 409. El override `docker-compose.chaos.yml` arma el proceso y hay que pedirlo de forma explícita. Armar tampoco inyecta nada: hace falta un segundo `POST` con las cabeceras `X-Lab-Token` y `X-Lab-Enable` en true. El token se compara y no queda en el log. `POST /api/chaos/stop` apaga ambas inyecciones sin reconstruir la imagen. Seguimos la idea de Rosenthal y Jones (2020) de que un experimento seguro se diseña para que el estado por defecto no haga daño.

8.1 Experimento de latencia

La espera es de 200 ms constantes dentro de `fulfillment.prepare`. El contador `chaos.latency.injections` y el campo `latencyInjectedMs` de la respuesta separan esa espera de la latencia normal. `service-a` se alarga porque espera a `service-b`, mientras que `data-service` no duerme. El efecto esperado sobre el SLO es cruzar los 150 ms de p99 y encender `ServiceALatencySlo`, y la traza debería mostrar `chaos.delay_ms=200`. Nada de eso lo vimos en contenedores. Lo que sí comprobó la prueba unitaria de `service-b` es que, con el proceso armado, la llamada espera al menos 180 ms y devuelve 200, y que desarmado no espera.

8.2 Experimento de error del 10 %

La probabilidad es una constante de 0,10 y solo aplica si el proceso está armado, el interruptor de errores está encendido y la petición trae `X-Lab-Experiment: error-rate`. Una petición sin esa cabecera no entra en el sorteo. El fallo ocurre antes del repositorio, así que no es un error de SQL disfrazado. La prueba unitaria fuerza el aleatorio: con un valor menor que 0,10 el servicio falla sin llamar a la base, y con un valor mayor o igual no falla. En una muestra corta de veinte peticiones el porcentaje observado puede alejarse bastante del 10; lo que defendemos es la probabilidad configurada, no el recuento de una muestra pequeña.

8.3 Qué contamos como detección

`chaos/run_scenarios.py` toma `started_at` al iniciar el escenario. La primera degradación observable es una respuesta HTTP 500 o una duración de al menos 180 ms, y la detección es el instante en que Prometheus muestra la alerta esperada en firing. `mttd_seconds` es la diferencia. Si la alerta no aparece en 150 segundos, el campo queda en null; no lo reemplazamos por 120. El objetivo de diseño es un MTTD de dos minutos o menos para la alerta estática, cuyo `for` es de 30 segundos y cuya ventana de rate es de 2 minutos. La alerta correlacionada necesita unos diez minutos de historia, así que no es la que usaríamos para cumplir los dos minutos. El script no se ejecutó y el MTTD sigue pendiente.

8.4 SLO y presupuesto: la única cifra calculada

El SLI de éxito es la fracción de respuestas 2xx de `/api/orders/` sobre todas las respuestas de esa ruta, y el SLO es 99 %. El presupuesto de una ventana es el 1 % de sus peticiones, y la fracción consumida son los fallos divididos entre ese

presupuesto (Beyer et al., 2016). La extrapolación es otra cosa: multiplica el consumo del experimento por su duración y lo divide entre la ventana del SLO.

`ErrorBudgetTest` fija 1000 peticiones, 10 fallos y SLO de 0,99. El presupuesto es de 10 peticiones y la fracción consumida es 1,0, es decir, esa ventana hipotética agota el presupuesto. Si se extrapolan 120 segundos a 30 días, la proporción es 120 / 2 592 000. La prueba demuestra la aritmética, no que un experimento de este laboratorio haya producido 10 fallos ni que el SLO se haya degradado en un entorno corriendo. Cuando exista `evidence/chaos/resultados.json`, el consumo se calculará con los totales de esa corrida y se etiquetará como ventana experimental, aparte de la extrapolación a 30 días.

Magnitud	Valor	Naturaleza
SLO de éxito	99 % de 2xx	Definición
Umbral de latencia	150 ms	Diseño; no medido en ejecución
Latencia inyectada	200 ms	Constante de código, comprobada en unidad
Probabilidad de error	0,10, solo con cabecera	Constante de código, comprobada en unidad
Consumo en el test	1,0 para 10 fallos de 1000	Prueba de la fórmula, no una corrida
MTTD	Pendiente	El script no se ejecutó
Reducción de ruido	Pendiente	No hay conteo de alertas en firing

Tabla 8. Definiciones, pruebas unitarias y mediciones que todavía faltan (elaboración propia).

La recuperación prevista es el `POST` de parada y, si hiciera falta, volver al compose sin el override para dejar `CHAOS_ARMED` en false. Después, un pedido ord-001 debe responder 200 con `latencyInjectedMs` en 0. Esa comprobación también depende de tener el stack corriendo. Los experimentos no deben lanzarse contra un servicio productivo ni contra una base cloud sin una cuenta de laboratorio y un alcance revisado, y en este ciclo no se lanzaron.

9. Evaluación de madurez

Buscamos un documento oficial llamado Observability Foundation Blueprint con una lista cerrada de ocho dominios y no lo encontramos. Por eso usamos la escala de 1 a 5 de la actividad (1 inicial, 2 básico y reactivo, 3 estandarizado y repetible, 4 avanzado y correlacionado, 5 optimizado y automatizado) sobre ocho dominios que definimos nosotros. No es una clasificación de ningún fabricante, y un YAML versionado no sube la nota.

Dominio	Nivel	Evidencia y techo
Telemetría de aplicación	3	Agente, spans y logs están en el código y las pruebas pasan. El pipeline no se vio corriendo por el 504 del registro.
Correlación	3	Derived fields, `tracesToLogsV2` y el script de enriquecimiento están escritos. Falta el salto en Grafana con un `trace_id` real.
Detección	2	Las reglas existen, pero ninguna alerta llegó a firing.
SRE	2	SLO, presupuesto y fórmula probados en unidad. No hay ventana experimental.
Seguridad y red	2	El 401 y el contador son código real. Flow logs y centros de seguridad cloud no están desplegados. No hay CVE.
AIOps	2	La regla estadística está versionada. No hubo conteo de alertas estáticas frente a correlacionadas.
Caos y recuperación	2	La política está probada. El MTTD contra Prometheus no se midió.
Plataforma multicloud	2	Terraform, manifiestos y perfiles en el repositorio. No hubo apply.

Tabla 9. Ningún dominio llega a 4 o 5: la correlación diseñada no reemplaza una traza observada.

Dejar la nota donde está es menos riesgoso que inflarla. Un nivel 4 exigiría haber seguido un incidente real de este sistema, con alerta, traza y recuperación medidas. Los riesgos concretos hoy son otros dos: creer que el compose está sano solo porque los archivos existen y descubrir el 504 del registro el día de la sustentación, y aplicar Cloud SQL o RDS sin revisar el costo y el emparejamiento de red.

10. Roadmap de 90 días

El orden prioriza detección, diagnóstico y recuperación. El mesh y los flow logs van después porque no destraban la primera evidencia. Cada quincena cabe en el repositorio actual sin reescribir los servicios.

Quincena	Acción	Prioridad	Criterio de éxito
1	Levantar el compose cuando el registro entregue las imágenes Java y guardar una traza con los tres servicios y el span JDBC.	Alta	El mismo `trace_id` en la respuesta, en Loki y en Tempo
2	Ejecutar `chaos/run_scenarios.py` y archivar el JSON.	Alta	`mttd_seconds` numérico, o null explícito si la alerta no apareció
3	Separar los tres procesos ECS y activar Service Connect en un namespace de laboratorio.	Media	Métricas L7 del proxy distintas de las del agente, en una cuenta autorizada
4	Activar Cloud Service Mesh con el fleet ya creado y comparar sus métricas con las del agente.	Media	Pods en Ready y una serie L7 que no sea `http_client` del agente
5	Encender VPC Flow Logs en la cuenta de laboratorio y armar un panel de rechazos con esa fuente.	Media	Registros de flujo reales, no una consulta vacía presentada como hallazgo
6	Revisar el umbral de 150 ms con el p99 medido en la quincena 1 y alargar el baseline si la desviación oscila.	Alta	Umbral justificado con un percentil de este laboratorio

Tabla 10. Seis quincenas. El éxito de la segunda es un archivo, no una estimación.

Las dependencias son claras. La quincena 1 depende de Docker Hub o de tener las imágenes ya cargadas en la máquina. La 2 depende de la 1 y de varios minutos de tráfico sano, porque la alerta correlacionada pide historia. La 3 y la 4 necesitan permisos y un tope de gasto. La 5 depende de la red que crea Terraform, y la 6, de la traza y las series de la quincena 1. Las dos primeras caben en una sesión de laboratorio si el registro responde; las siguientes son trabajo de días, porque incluyen revisión de IAM y de costo.

11. Conclusiones y recomendaciones

El laboratorio deja un sistema coherente: una sola base por proceso, tres spans de negocio que no repiten el SQL, un caos con dos compuertas y una regla de anomalía que no se dispara con una serie recién nacida. Las pruebas unitarias respaldan esas compuertas y la aritmética del presupuesto. Con eso alcanza para defender el diseño, pero no para defender un tiempo de detección, una captura o un ahorro de alertas.

Descartar App Mesh tiene fecha: el fin de soporte del 30 de septiembre de 2026. Reemplazarlo por un mesh sin aplicar habría sido peor que reconocer que, hoy, la única observabilidad L7 del diseño local es la del agente. Con los CVE y los centros de seguridad aplicamos el mismo criterio: si no hay fuente, no hay panel numérico.

Recomendaciones, en el orden en que destraban la sustentación:

1. Repetir `docker compose` cuando el registro responda y comprobar ord-001, el `traceId` compartido y el span JDBC antes de grabar.
2. Armar el caos solo con el override, ejecutar los dos experimentos, pararlos y dejar un pedido sano. Si el 504 se repite, no grabar el tramo de contenedores.
3. Ejecutar `chaos/run_scenarios.py` y conservar el JSON aunque `mttd_seconds` salga null.
4. No calcular ningún porcentaje de ruido hasta tener los tres escenarios en ese archivo.

5. No hacer `terraform apply` en el ensayo. Si después hay cuenta, empezar por plan con `create_cloud_sql` y `create_rds` en false.
6. Revisar el umbral de 150 ms apenas exista un p99 de este stack, y mientras tanto presentarlo como umbral de diseño.

Lo que más nos llevamos es de método. Una alerta enriquecida se diseña con las señales que de verdad existen: el `trace_id` vive en el log, no en el histograma. Un experimento seguro se diseña para que el estado por defecto no haga daño. Y la nota de madurez se escribe después de la corrida, no antes.

Archivos del repositorio que sostienen el informe: `deploy/local/docker-compose.yaml`, `observability/collector/collector-local.yaml`, `observability/prometheus/rules/lab33-alerts.yml`, `observability/aiops/enrich_incident.py`, `chaos/run_scenarios.py`, `docs/arquitectura.md`, `docs/madurez.md` y `PASOS.md`.

Referencias

Amazon Web Services. (s.f.-a). *What is AWS App Mesh?* Recuperado el 9 de octubre de 2026, de https://docs.aws.amazon.com/app-mesh/latest/userguide/what-is-app-mesh.html

Amazon Web Services. (s.f.-b). *Use Service Connect to connect Amazon ECS services with short names*. Recuperado el 9 de octubre de 2026, de https://docs.aws.amazon.com/AmazonECS/latest/developerguide/service-connect.html

Amazon Web Services. (s.f.-c). *Logging IP traffic using VPC Flow Logs*. Recuperado el 9 de octubre de 2026, de https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html

Beyer, B., Jones, C., Petoff, J. y Murphy, N. R. (Eds.). (2016). *Site reliability engineering: How Google runs production systems*. O'Reilly Media. https://sre.google/sre-book/table-of-contents/

Google Cloud. (s.f.-a). *Cloud Service Mesh overview*. Recuperado el 9 de octubre de 2026, de https://cloud.google.com/service-mesh/docs/overview

Google Cloud. (s.f.-b). *VPC Flow Logs*. Recuperado el 9 de octubre de 2026, de https://cloud.google.com/vpc/docs/flow-logs

Grafana Labs. (s.f.-a). *Provision Grafana*. Recuperado el 9 de octubre de 2026, de https://grafana.com/docs/grafana/latest/administration/provisioning/

Grafana Labs. (s.f.-b). *Grafana Tempo documentation*. Recuperado el 9 de octubre de 2026, de https://grafana.com/docs/tempo/latest/

Grafana Labs. (s.f.-c). *Grafana Loki documentation*. Recuperado el 9 de octubre de 2026, de https://grafana.com/docs/loki/latest/

OpenTelemetry. (s.f.-a). *Java agent*. Recuperado el 9 de octubre de 2026, de https://opentelemetry.io/docs/zero-code/java/agent/

OpenTelemetry. (s.f.-b). *Collector*. Recuperado el 9 de octubre de 2026, de https://opentelemetry.io/docs/collector/

OpenTelemetry. (s.f.-c). *Semantic conventions*. Recuperado el 9 de octubre de 2026, de https://opentelemetry.io/docs/specs/semconv/

Prometheus. (s.f.). *Query functions*. Recuperado el 9 de octubre de 2026, de https://prometheus.io/docs/prometheus/latest/querying/functions/

Rosenthal, C. y Jones, N. (2020). *Chaos engineering: System resiliency in practice*. O'Reilly Media.

W3C. (2021). *Trace Context* (W3C Recommendation). https://www.w3.org/TR/trace-context/